

Practical 11 - Shrinkage and variable selection - ridge and lasso regression

Statistical Models for Discrete Outcomes

London School of Hygiene & Tropical Medicine

SOLUTIONS

Datasets required: `SAheart.csv`

In this practical we will explore ridge and lasso regression using the South Africa data used in the previous practical.

Question 1

Load the dataset and fit the same logistic regression model as in the previous practical for the `chd` outcome, with covariates `sbp tobacco ldl famhist obesity alcohol age`, saving the estimates into an object called `logitmle`.

SOLUTIONS

```
SAheart <- read.csv("SAheart.csv", row.names = 1)
logitmle <- glm(chd~sbp+tobacco+ldl+factor(famhist)+
               obesity+alcohol+age,
               data=SAheart,
               family="binomial")
```

We will next fit the model using ridge regression, using the R package `glmnet` (which you will need to install).

Question 2

Run the following code to fit the model using ridge regression. You will need to examine the documentation for `glmnet` to find out what value of `alpha` you need for ridge regression.

```
library(glmnet)
ridge_fit <- glmnet(model.matrix(logitmle),
                    SAheart$chd, family = "binomial",
                    alpha=)
```

SOLUTIONS

The documentation for `glmnet` explains that the function uses what is termed an ‘elastic net penalty’:

$$(1 - \alpha)/2 \|\beta\|_2^2 + \alpha \|\beta\|_1$$

with α a value that we specify. Here $\|\cdot\|_2$ and $\|\cdot\|_1$ are the Euclidean norm and Manhattan/absolute value norms (see [Wikipedia](#)). From the definitions of these ‘norm’ functions and more directly from what is written in the `glmnet` documentation, setting $\alpha = 0$ gives us ridge regression. We thus run:

```
library(glmnet)
ridge_fit <- glmnet(model.matrix(logitmle),
                    SAheart$chd, family = "binomial",
                    alpha=0)
```

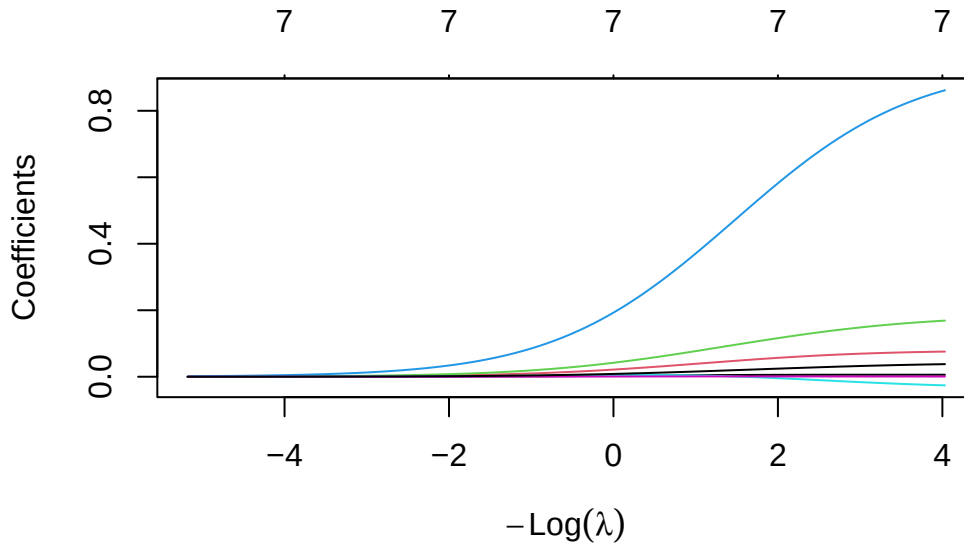
Question 3

Run `plot(ridge_fit)`. What is the plot showing? Take a look at `?plot.glmnet` to see how you can change the plot.

SOLUTIONS

If we plot `ridge_fit` we obtain:

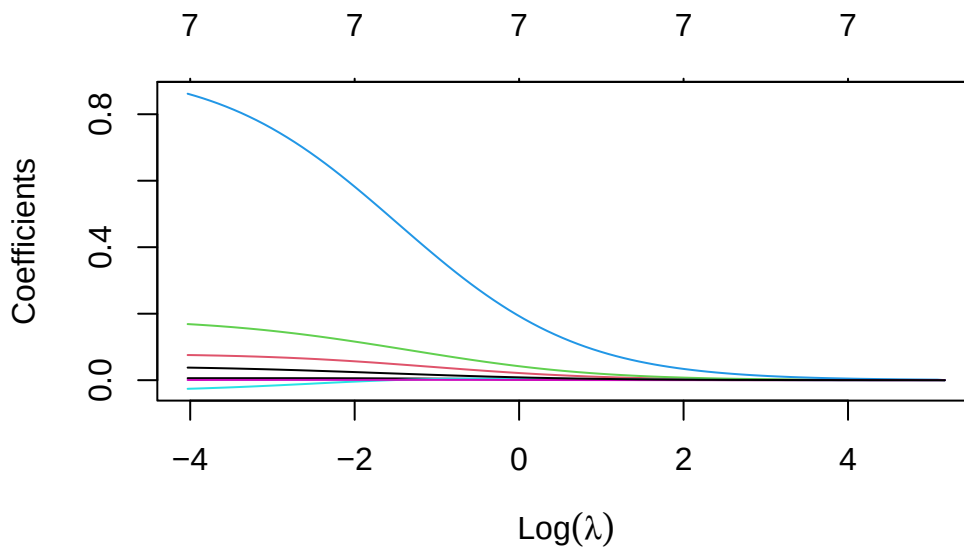
```
plot(ridge_fit)
```



On the y-axis are the coefficient values of the covariates at each value of λ . `glmnet` has refitted the model using a range of different λ values, and is plotting the estimated coefficients of each (standardised) covariate for that λ value.

The x-axis has default to show $-\log(\lambda)$. Higher values of λ result in lower values of $-\log(\lambda)$, which is why the coefficients are being shrunk towards zero on the left hand side of the plot, but getting larger on the right.

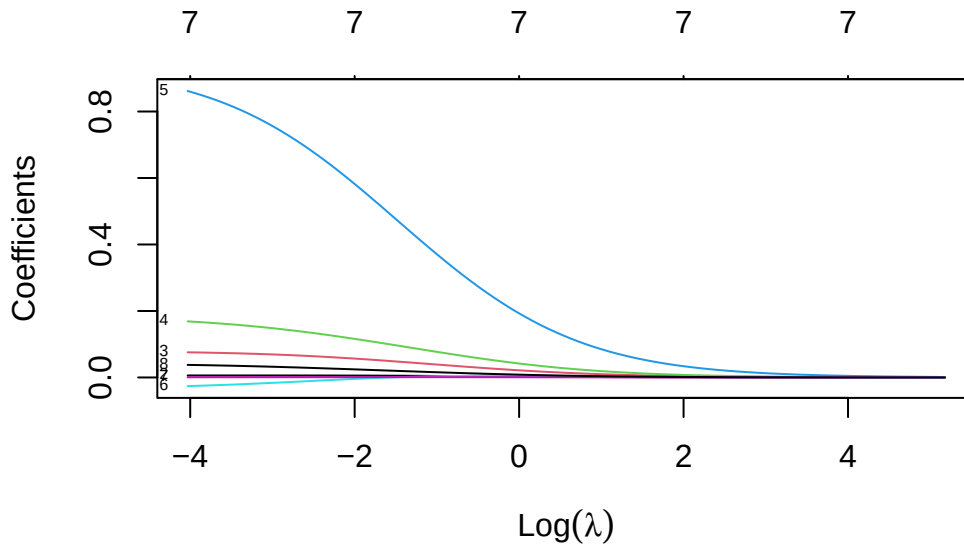
```
plot(ridge_fit,sign.lambda=1)
```



which just switches the x-axis to $\log(\lambda)$.

To see which curve corresponds to which covariate, with the number corresponding to the column of the model matrix we passed to `glmnet`:

```
plot(ridge_fit, sign.lambda=1,
     label=TRUE)
```



The numbers on the top of the plot are the number of non-zero coefficient at the corresponding value of λ . Because we have used ridge regression, none of the coefficients get shrunk completely to zero, which is why there are 7 non-zero coefficients for all λ values it has used.

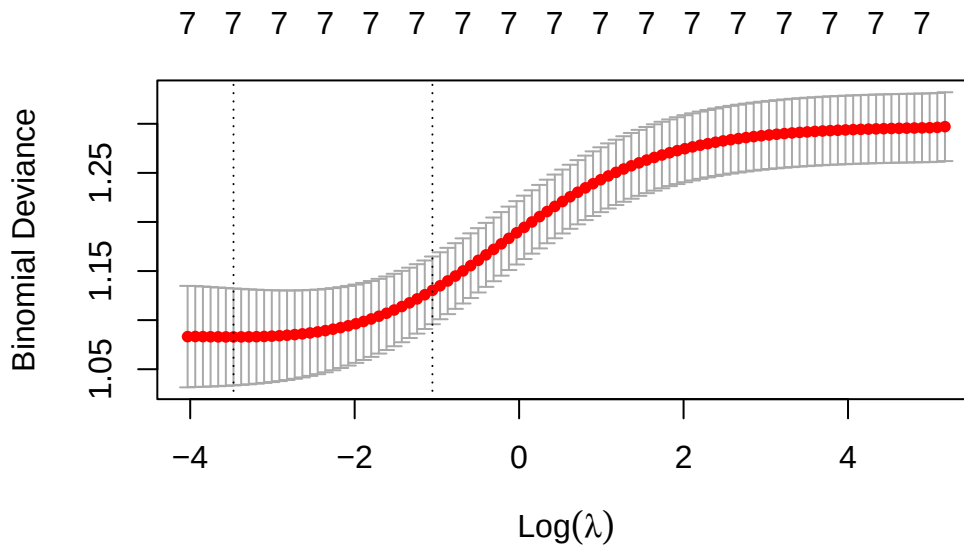
Question 4

Use the `cv.glmnet` function to perform 10 fold cross-validation for the ridge regression. If you want to reproduce the results in the solutions, use `set.seed(789)` before you run `cv.glmnet`.

Plot the object that `cv.glmnet` returns. What does the plot show? Page 5 of the `glmnet` package [vignette](#) gives further details.

SOLUTIONS

```
set.seed(789)
cvfit <- cv.glmnet(model.matrix(logitmlle),
                  SAheart$chd, family = "binomial",
                  alpha=0)
plot(cvfit, sign.lambda = 1)
```



The plot shows how the cross-validation estimate of deviance changes with different values of λ . Smaller values of the deviance mean better fit. The optimal value of λ , that is the value when the deviance is estimated to be lowest, is shown by the left hand dashed vertical line. This value of λ can be extracted using

```
cvfit$lambda.min
```

```
[1] 0.03101155
```

```
log(cvfit$lambda.min)
```

```
[1] -3.473395
```

The other dashed line is the largest value of λ (or rather $\log(\lambda)$) for which the deviance is no more than one standard error larger than the deviance of the deviance at the best value of λ . The logic of this is that the cross-validation estimate of deviance against λ is estimated with error, and so if anything we err on the side of a more regularised/penalised model (see P61 of [The Elements of Statistical Learning](#)).

Question 5

Examine the documentation for `coef.glmnet`. Call this function with your cross-validation object from the previous question to examine the estimated coefficients at the value of λ found to be the best from cross-validation. Compare the coefficients with those from question 1.

SOLUTIONS

We can obtain the coefficients (on the original scale of the covariates) at the value of λ estimated to give the lowest deviance with

```
coef.glmnet(cvfit, s=cvfit$lambda.min)
```

```
9 x 1 sparse Matrix of class "dgCMatrix"
              s=0.03101155
(Intercept)  -3.9510954973
(Intercept)      .
sbp           0.0060078120
tobacco       0.0729047781
ldl           0.1592570504
factor(famhist)Present 0.8138777478
obesity      -0.0212667323
alcohol       0.0006850874
age           0.0351734964
```

If we then recall the coefficients from the original MLE fit:

```
coef(logitmle)
```

(Intercept)	sbp	tobacco
-4.1295996883	0.0057606767	0.0795256305
ldl	factor(famhist)Present	obesity
0.1847793334	0.9391854851	-0.0345434340
alcohol	age	
0.0006065017	0.0425412093	

we can see that the coefficients of most of the covariates have been shrunk somewhat towards zero. This is the effect of penalisation/regularisation.

Question 6

Examine the documentation for `predict.cv.glmnet`. Then use `predict` applied to your cross-validation object to create the fitted probability values of the CHD outcome when λ is set to the value found to be best by cross-validation. When calling `predict` you can use `type="response"` to request fitted probabilities.

Compare the fitted values from ridge regression at this value of λ to the fitted values from the MLE logistic regression in question 1.

SOLUTIONS

```
ridgePred <- predict(cvfit, newx=model.matrix(logitmle),  
                    s="lambda.min",  
                    type="response")  
mlePred <- logitmle$fitted.values  
summary(ridgePred)
```

```
      lambda.min  
Min.      :0.03982  
1st Qu.:0.16616  
Median :0.32121  
Mean    :0.34632  
3rd Qu.:0.49999  
Max.    :0.86447
```

```
sd(ridgePred)
```

```
[1] 0.2067616
```

```
summary(mlePred)
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.  
0.02175 0.14580 0.31316 0.34632 0.52084 0.88687
```

```
sd(mlePred)
```

```
[1] 0.2274257
```

We see that the mean fitted probabilities are the same, but the minimum and maximum are larger in magnitude from the MLE. Also the SD of the fitted probabilities from the ridge regression are smaller - the penalised ridge regression leads to less extreme predicted probabilities, as a result of the shrunken effects of the covariates.

Question 7

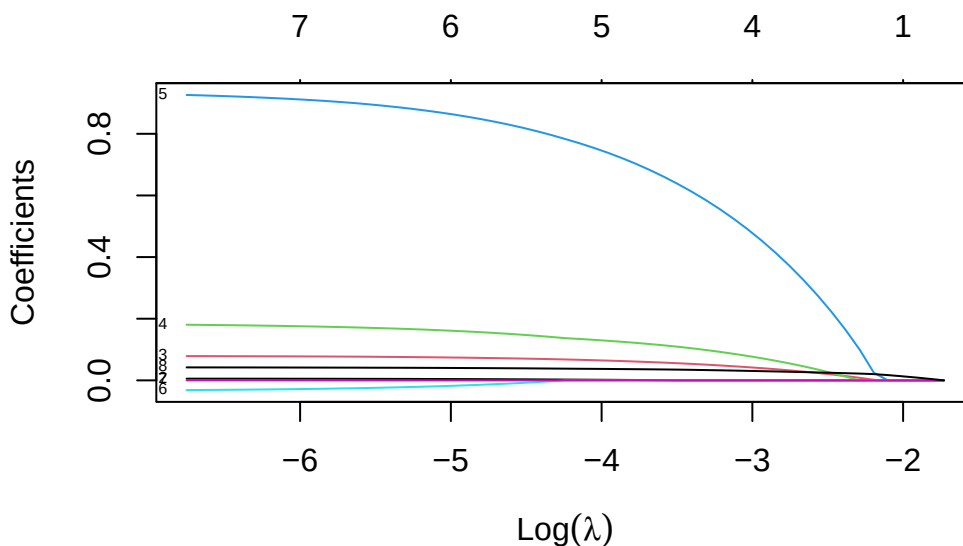
We will now explore lasso regression.

Fit the same logistic model using lasso regression with `glmnet`. Produce the plot of coefficients against λ . What has changed compared to the earlier ridge regression?

SOLUTIONS

Recalling earlier the meaning of the argument α to `glmnet`, we now re-run with `alpha=1`:

```
lasso_fit <- glmnet(model.matrix(logitmle),  
                    SAheart$chd, family = "binomial",  
                    alpha=1)  
plot(lasso_fit, sign.lambda=1,  
     label=TRUE)
```



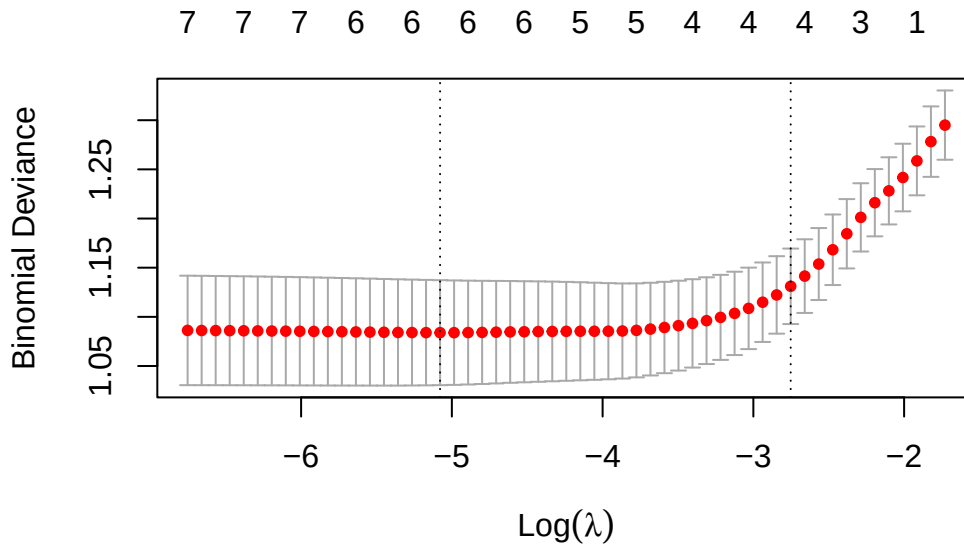
The shape of the coefficient plot curves is somewhat different to what we saw earlier in the ridge regression. Moreover, as indicated by the numbers at the top of the plot, some of the coefficients are now being shrunk to precisely zero as λ is increased.

Question 8

Use cross-validation for lasso with 10 folds and `set.seed(789)` to find the value of λ that minimizes the deviance. Use `coef.glmnet()` to find which covariates have been retained at this value of λ , and compare with the earlier MLE and ridge regression estimated coefficients.

SOLUTIONS

```
set.seed(789)
cvfitlasso <- cv.glmnet(model.matrix(logitmle),
  SAheart$chd, family = "binomial",
  alpha=1)
plot(cvfitlasso, sign.lambda = 1)
```



We see that there are 6 non-zero coefficients at the (estimated) optimal value of λ . These are:

```
coef.glmnet(cvfitlasso, "lambda.min")
```

```
9 x 1 sparse Matrix of class "dgCMatrix"
      lambda.min
(Intercept) -4.065386471
(Intercept) .
sbp          0.004379621
tobacco      0.074587069
ldl          0.162773895
factor(famhist)Present 0.869609402
obesity      -0.019021880
alcohol      .
age          0.040376054
```

```
#note you can also get the same result with
#coef.glmnet(cvfitlasso, "lambda.min")
```

The variable dropped is `alcohol1`. The lasso coefficients are shrunk towards zero compared to the MLEs. Comparing back with the ridge regression coefficients, some are larger and some are smaller (in absolute magnitude).

Question 9

Use the following code (from the last practical) to split the `SAheart` dataset into training and test subsets:

```
set.seed(592723)
n <- dim(SAheart)[1]
train_idx <- sample(seq_len(n), size = 0.75 * n)
trainData <- SAheart[train_idx, ]
testData <- SAheart[-train_idx, ]
```

Next, fit the logistic model using the *training data* with:

- MLE with `glm`
- Cross-validated ridge regression with `cv.glmnet`
- Cross-validated lasso regression with `cv.glmnet`

Then use `assess.glmnet` to assess the performance of the ridge and lasso models. Lastly, you can use the `pROC` package and code from the previous practical to assess the performance of the logistic model fitted by MLE. Compare the performance of all three.

SOLUTIONS

```
set.seed(592723)
n <- dim(SAheart)[1]
train_idx <- sample(seq_len(n), size = 0.75 * n)
trainData <- SAheart[train_idx, ]
testData <- SAheart[-train_idx, ]

trainMLE <- glm(chd~sbp+tobacco+ldl+factor(famhist)+
               obesity+alcohol+age,
               data=trainData,
               family="binomial")
trainRidge <- cv.glmnet(model.matrix(trainMLE),
                       trainData$chd, family = "binomial",
                       alpha=0)
trainLasso <- cv.glmnet(model.matrix(trainMLE),
                       trainData$chd, family = "binomial",
```

```

        alpha=1)

#now assess the models
#fit MLE model to test data to get model matrix
testMLE <- glm(chd~sbp+tobacco+ldl+factor(famhist)+
              obesity+alcohol+age,
              data=testData,
              family="binomial")
assess.glmnet(trainRidge, newx=model.matrix(testMLE),
              newy=testData$chd)

```

```

$deviance
lambda.1se
  1.125154
attr(,"measure")
[1] "Binomial Deviance"

$class
lambda.1se
  0.2844828
attr(,"measure")
[1] "Misclassification Error"

$auc
[1] 0.7590244
attr(,"measure")
[1] "AUC"

$mse
lambda.1se
  0.3798347
attr(,"measure")
[1] "Mean-Squared Error"

$mae
lambda.1se
  0.8000189
attr(,"measure")
[1] "Mean Absolute Error"

```

```
assess.glmnet(trainLasso, newx=model.matrix(testMLE),  
              newy=testData$chd)
```

```
$deviance  
lambda.1se  
  1.100876  
attr(,"measure")  
[1] "Binomial Deviance"  
  
$class  
lambda.1se  
  0.2844828  
attr(,"measure")  
[1] "Misclassification Error"  
  
$auc  
[1] 0.764878  
attr(,"measure")  
[1] "AUC"  
  
$mse  
lambda.1se  
  0.3699815  
attr(,"measure")  
[1] "Mean-Squared Error"  
  
$mae  
lambda.1se  
  0.7791109  
attr(,"measure")  
[1] "Mean Absolute Error"
```

Various performance measures are reported by `assess.glmnet`. From the output we can see `lambda.1se` being mentioned - this is because by default it is reporting model performance measures based on the value of λ that is one SE larger than the optimal value, mentioned earlier. The AUC for ridge is estimated at 0.759 and for lasso at 0.765, so slightly higher for the lasso.

Note that lasso does not perform generally better than ridge in general - which is expected to perform better would depend on whether some of the coefficients are in truth zero, in which case we would expect lasso to do better.

Lastly, we can use the test data to evaluate the performance of the MLE fit of the logistic model, using the pROC package as in the previous practical:

```
library(pROC)
roc(testData$chd, predict(trainMLE,
                           newdata = testData,
                           type="response"))
```

Call:

```
roc.default(response = testData$chd, predictor = predict(trainMLE,      newdata = testData, t
```

```
Data: predict(trainMLE, newdata = testData, type = "response") in 75 controls (testData$chd (
Area under the curve: 0.7704
```

The estimated AUC is 0.770 - higher than both the ridge and lasso! Remember though that these performance measures are estimates based on just 116 observations. A larger test dataset would enable us to obtain more precise estimates of performance.

Another possible explanation is that as noted earlier, `assess.glmnet` has given us model performance at the `lambda.1se` value of λ . We can instead ask it for the performance at the value of λ found to be optimal:

```
#ridge
assess.glmnet(trainRidge, newx=model.matrix(testMLE),
              newy=testData$chd, s="lambda.min")
```

```
$deviance
lambda.min
  1.083319
attr(,"measure")
[1] "Binomial Deviance"

$class
lambda.min
  0.2758621
attr(,"measure")
[1] "Misclassification Error"

$auc
[1] 0.7655285
attr(,"measure")
```

```
[1] "AUC"
```

```
$mse
```

```
lambda.min
```

```
0.3635728
```

```
attr(,"measure")
```

```
[1] "Mean-Squared Error"
```

```
$mae
```

```
lambda.min
```

```
0.7398155
```

```
attr(,"measure")
```

```
[1] "Mean Absolute Error"
```

```
#lasso
```

```
assess.glmnet(trainLasso, newx=model.matrix(testMLE),  
              newy=testData$chd, s="lambda.min")
```

```
$deviance
```

```
lambda.min
```

```
1.083465
```

```
attr(,"measure")
```

```
[1] "Binomial Deviance"
```

```
$class
```

```
lambda.min
```

```
0.2844828
```

```
attr(,"measure")
```

```
[1] "Misclassification Error"
```

```
$auc
```

```
[1] 0.7658537
```

```
attr(,"measure")
```

```
[1] "AUC"
```

```
$mse
```

```
lambda.min
```

```
0.3630786
```

```
attr(,"measure")
```

```
[1] "Mean-Squared Error"
```

```
$mae
```

```
lambda.min  
0.7342217  
attr(,"measure")  
[1] "Mean Absolute Error"
```

The AUC values have increased slightly compared to the values using `lambda.1se`, but they are still lower than the MLE AUC.